Лабораторна робота № 5

Тема: Побудова простої RAG-системи

Мета роботи: Ознайомитися з принципом роботи Retrieval-Augmented Generation (RAG); навчитися використовувати зовнішню базу документів для пошуку інформації та формування відповідей великою мовною моделлю; набути практичних навичок роботи з embeddings, векторним сховищем та пошуком релевантних фрагментів тексту.

Короткі теоретичні відомості

RAG (Retrieval-Augmented Generation) — підхід, за якого мовна модель отримує додаткову інформацію з зовнішнього джерела перед формуванням відповіді.

Embedding — числове представлення тексту у вигляді вектора, яке дозволяє порівнювати тексти за їхнім змістом.

Vector Store — сховище, у якому зберігаються embedding-вектори документів та інформація, необхідна для пошуку.

Retrieval — пошук у базі документів фрагментів, найбільш релевантних до запиту користувача.

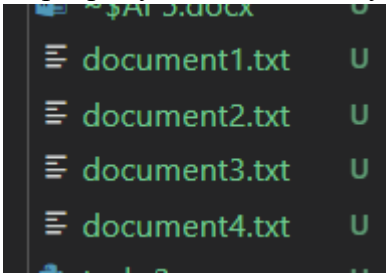
Опис використаних технологій

Модель ембеддінгів: gemini-embedding-001

Модель генерації: gemini-3.5-flash-lite

Векторна база даних: ChromaDB

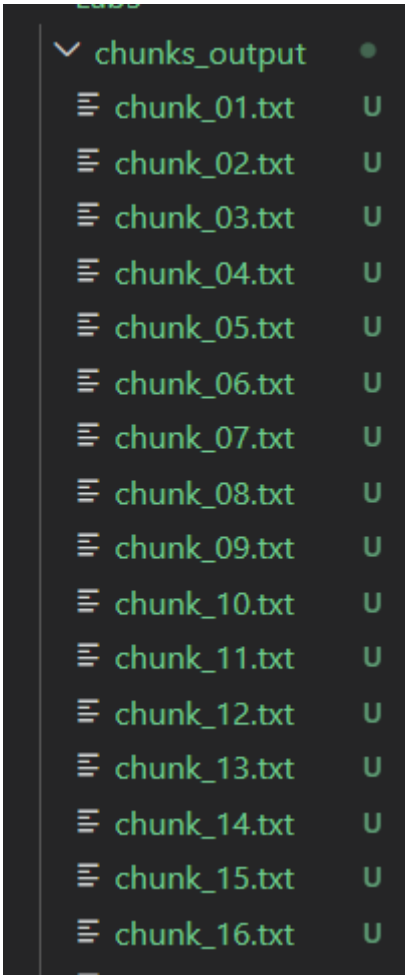
Завдання 1
Для створення бази знань підготовлено 4 текстові документи у форматі .txt з теми «Програмування та мова Python»:



Завдання 2

Підготовка документів Завантажити документи в програму та розділити їх на невеликі фрагменти (chunks).

```
Успішно створено та збережено 20 файлів у папку:
e:\лабораторні 3 курс\AI\Lab5\chunks_output
```




```python
import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

file_names = ['document1.txt', 'document2.txt', 'document3.txt', 'document4.txt']

chunks = []
chunk_id = 1

for file_name in file_names:
    file_path = os.path.join(BASE_DIR, file_name)
    if os.path.exists(file_path):
        with open(file_path, 'r', encoding='utf-8') as f:
            text = f.read()

            raw_chunks = [p.strip() for p in text.split('\n\n') if p.strip()]

            for paragraph in raw_chunks:
                chunks.append({
                    "id": f"chunk_{chunk_id:02d}",
                    "content": paragraph,
                    "source": file_name
```

```
                })
                chunk_id += 1

output_dir = os.path.join(BASE_DIR, "chunks_output")
os.makedirs(output_dir, exist_ok=True)

for item in chunks:
    file_name = f"{item['id']}.txt"
    file_path = os.path.join(output_dir, file_name)

    with open(file_path, 'w', encoding='utf-8') as f:
        f.write(f"Source: {item['source']}\n")
        f.write(f"ID: {item['id']}\n")
        f.write("-" * 40 + "\n\n")
        f.write(item['content'])

print(f"Успішно створено та збережено {len(chunks)} файлів у папку:\n{output_dir}")
```

Завдання 3. Створення embeddings

Для кожного фрагмента тексту створити embedding за допомогою обраної embedding-моделі.

Завдання 4. Створення векторного сховища

```
import os
import chromadb
from google import genai

client = genai.Client(api_key=API_KEY)

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
CHUNKS_DIR = os.path.join(BASE_DIR, "chunks_output")
CHROMA_DB_DIR = os.path.join(BASE_DIR, "chroma_db")

def get_embedding(text: str) -> list[float]:
    response = client.models.embed_content(
        model="gemini-embedding-001",
        contents=text,
    )
    return response.embeddings[0].values

print("Завдання 3: Генерація Embeddings ")

ids = []
documents = []
metadatas = []
embeddings = []

if os.path.exists(CHUNKS_DIR):
    chunk_files = sorted(os.listdir(CHUNKS_DIR))
```

```
    for file_name in chunk_files:
        if file_name.endswith(".txt"):
            file_path = os.path.join(CHUNKS_DIR, file_name)

            with open(file_path, "r", encoding="utf-8") as f:
                lines = f.readlines()

                source = lines[0].replace("Source:", "").strip()
                chunk_id = lines[1].replace("ID:", "").strip()
                content = "".join(lines[4:]).strip()

                print(f"Генерація вектора для {chunk_id} ({source})...")

                try:
                    vector = get_embedding(content)

                    ids.append(chunk_id)
                    documents.append(content)
                    metadatas.append({"source": source})
                    embeddings.append(vector)
                except Exception as e:
                    print(f"Помилка створення вектора для {chunk_id}: {e}")

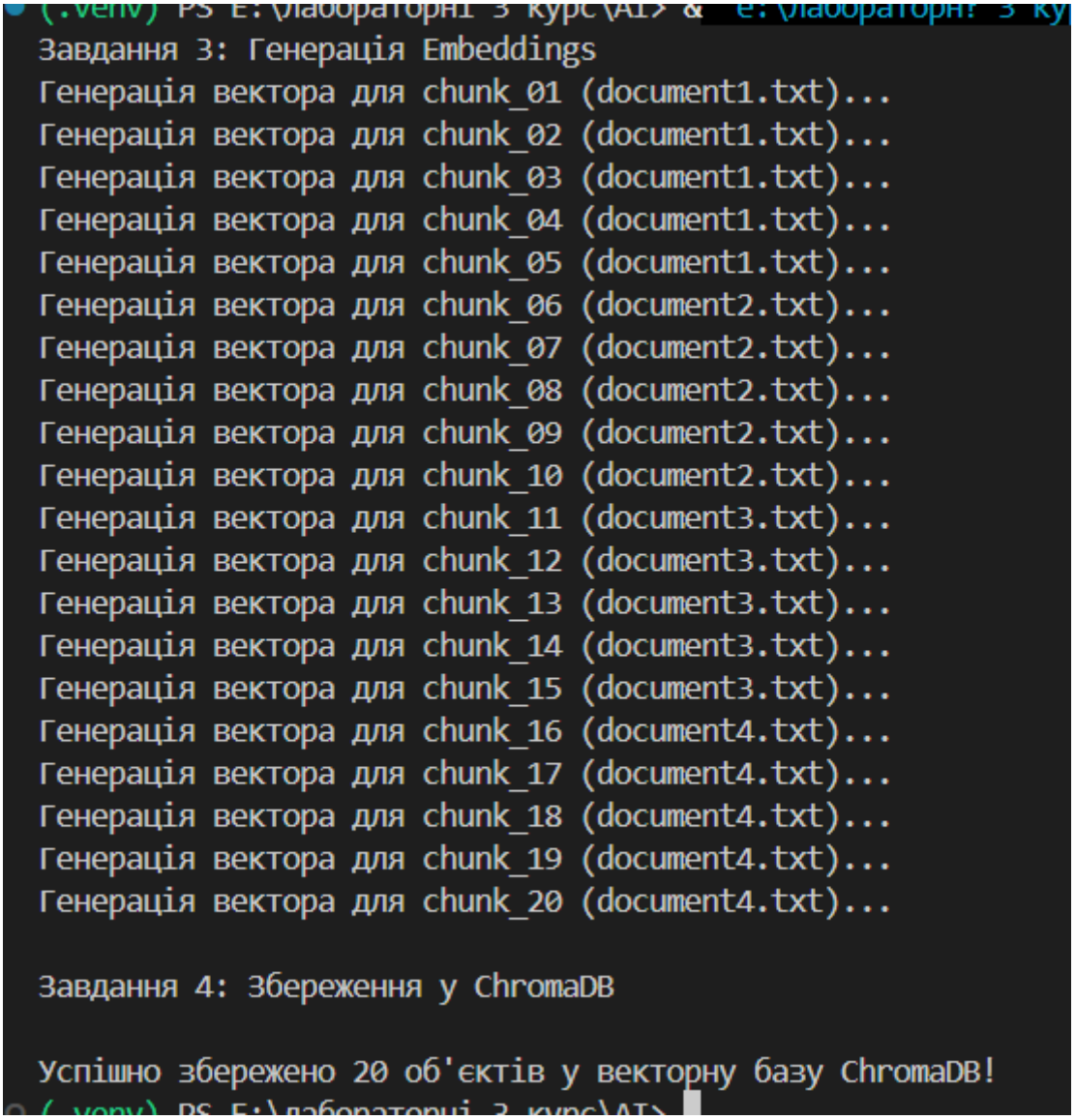

print("\nЗавдання 4: Збереження у ChromaDB")

chroma_client = chromadb.PersistentClient(path=CHROMA_DB_DIR)

collection = chroma_client.get_or_create_collection(name="python_docs")

if ids:
    collection.add(
        ids=ids,
        embeddings=embeddings,
        documents=documents,
        metadatas=metadatas
    )

print(f"\nУспішно збережено {collection.count()} об'єктів у векторну базу
ChromaDB!")
```



Завдання 5. Реалізація пошуку

Завдання 6. Формування відповіді за допомогою LLM

Завдання 8. Дослідження обмежень RAG

```python
import os
import chromadb
from google import genai

API_KEY = "AQ.Ab8RN6JbKiWzuEsvy0vAhfU6N6sYsmphU_ltDjt8yqn1pVJZeA"
client = genai.Client(api_key=API_KEY)

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
CHROMA_DB_DIR = os.path.join(BASE_DIR, "chroma_db")

chroma_client = chromadb.PersistentClient(path=CHROMA_DB_DIR)
collection = chroma_client.get_collection(name="python_docs")
```

```
def get_embedding(text: str) -> list[float]:
    response = client.models.embed_content(
        model="gemini-embedding-001",
        contents=text,
    )
    return response.embeddings[0].values

def retrieve_dynamic_chunks(query: str, max_distance: float = 0.55, max_k: int =
5):
    query_vector = get_embedding(query)
    results = collection.query(
        query_embeddings=[query_vector],
        n_results=max_k
    )

    retrieved_chunks = []

    if results and results['documents'] and results['documents'][0]:
        documents = results['documents'][0]
        ids = results['ids'][0]
        metadatas = results['metadatas'][0]
        distances = results['distances'][0] if 'distances' in results else [0] *
len(ids)

        for idx, dist in enumerate(distances):
            if dist <= max_distance:
                retrieved_chunks.append({
                    "id": ids[idx],
                    "content": documents[idx],
                    "source": metadatas[idx]['source'],
                    "distance": dist
                })

        if not retrieved_chunks:
            retrieved_chunks.append({
                "id": ids[0],
                "content": documents[0],
                "source": metadatas[0]['source'],
                "distance": distances[0]
            })

    return retrieved_chunks

def answer_with_rag(query: str, max_distance: float = 0.55):
    chunks = retrieve_dynamic_chunks(query, max_distance=max_distance)

    context_text = "\n\n".join([f"[{c['source']} | {c['id']} | Distance:
{c['distance']:.4f}]:\n{c['content']}" for c in chunks])

    prompt = f"""Використовуючи наведений нижче контекст, надай чітку та точну
відповідь на запитання користувача.
Якщо відповіді на запитання немає в наданому контексті, обов'язково повідом про це.
```

```
Контекст:
{context_text}

Запитання:
{query}
"""

    response = client.models.generate_content(
        model="gemini-3.5-flash-lite",
        contents=prompt,
    )

    return response.text, chunks

if __name__ == "__main__":
    test_questions = [
        "Що таке Python і хто його створив?",
        "Що таке списки та кортежі та яка їх відмінність",
        "Що таке концепція Качиної типізації (Duck Typing) в ООП?",
        "Для чого використовуються фреймворки LangChain та LlamaIndex?",
        "Що таке векторні сховища і для чого призначені Chroma та FAISS?",
        "Як працюють модифікатори доступу public, private та protected у мові
C++?"  #додаькове питання для перевірки обмежень RAG
    ]

    for idx, q in enumerate(test_questions, 1):
        print(f"ЗАПИТ #{idx}: {q}")
        answer, retrieved_chunks = answer_with_rag(q, max_distance=0.55)

        print(f"\n[Знайдено релевантних чанків: {len(retrieved_chunks)}]")
        for c in retrieved_chunks:
            print(f"  • ID: {c['id']} | Джерело: {c['source']} ")

        print("\n[Відповідь LLM (Gemini)]:")
        print(answer)
```

Завдання 7. Перевірка роботи RAG-системи

№	Запит	Знайдений фрагмент	Джерело	Відповідь LLM
1	Що таке Python і хто його створив?	chunk_01	document1.txt	На основі наданого контексту: **Python** — це сучасна високорівнева мова програмування інтерпретованого типу, яка відзначається лаконічним і читабельним синтаксисом. **Творцем мови є Гвідо ван Россум**, який розробив її наприкінці 1980-х років.
2	Що таке списки та кортежі та яка їх відмінність	chunk_06 chunk_07	document2.txt	На основі наданого контексту: * **Списки (`list`)** — це впорядковані *змінні* колекції, які можуть містити елементи довільних

				типів даних. Вони створюються за допомогою квадратних дужок або функції `list` і є динамічними масивами, розмір яких автоматично збільшується при додаванні нових елементів. * **Кортежі (`tuple`)** — це впорядковані *незмінні* колекції, які оголошуються за допомогою круглих дужок. **Основна відмінність** між ними полягає в тому, що після створення елементи кортежу **не можна змінювати**, тоді як списки є змінними колекціями.
3	Що таке концепція Качиної типізації (Duck Typing) в ООП?	chunk_14	document3.txt	Згідно з наданим контекстом, концепція качиної типізації (Duck Typing) у Python — це спосіб реалізації поліморфізму, за якого об'єкт може використовуватися незалежно від свого конкретного типу, якщо він реалізує необхідні методи.
4	Для чого використовуються фреймворки LangChain та LlamaIndex?	chunk_18	document4.txt	Згідно з наданим контекстом: * **LangChain** надає зручні абстракції для побудови складних ланцюжків обробки природної мови та дозволяє об'єднувати мовні моделі, бази даних, векторні сховища й зовнішні API. * **LlamaIndex** спеціалізується на індексуванні та структуруванні даних для використання у RAG (Retrieval-Augmented Generation). Обидва фреймворки використовуються як інструменти для створення RAG-систем та роботи з LLM.
5	Що таке векторні сховища і для чого призначені Chroma та FAISS?	chunk_19	document4.txt	На основі наданого контексту: * **Векторні сховища (Vector Stores)** призначені для збереження векторів (векторних представлень тексту, отриманих за допомогою моделей ембеддінгів) та швидкого пошуку за схожістю. * **Chroma** — це проста у використанні векторна база даних з відкритим кодом, яка підходить для прототипування. * **FAISS** — це бібліотека від компанії Meta, оптимізована для надшвидкого пошуку найближчих векторів у пам'яті.
6	Як працюють модифікатори доступу public, private та protected у мові C++?	chunk_12	document3.txt	У наданому контексті немає відповіді на запитання про те, як працюють модифікатори доступу public, private та protected у мові C++.

Порівняти відповіді: 1. LLM без використання RAG та 2. LLM із використанням RAG

```
import os
```

```python
import chromadb
from google import genai

API_KEY = "AQ.Ab8RN6JbKiWzuEsvy0vAhfU6N6sYsmphU_ltDjt8yqn1pVJZeA"
client = genai.Client(api_key=API_KEY)

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
CHROMA_DB_DIR = os.path.join(BASE_DIR, "chroma_db")

chroma_client = chromadb.PersistentClient(path=CHROMA_DB_DIR)
collection = chroma_client.get_collection(name="python_docs")


def get_embedding(text: str) -> list[float]:
    response = client.models.embed_content(
        model="gemini-embedding-001",
        contents=text,
    )
    return response.embeddings[0].values

def answer_without_rag(query: str) -> str:
    response = client.models.generate_content(
        model="gemini-3.5-flash-lite",
        contents=query,
    )
    return response.text.strip()

def answer_with_rag(query: str, max_distance: float = 0.62) -> tuple[str, list]:
    query_vector = get_embedding(query)
    results = collection.query(query_embeddings=[query_vector], n_results=3)

    retrieved_chunks = []
    if results and results['documents'] and results['documents'][0]:
        for idx, dist in enumerate(results['distances'][0]):
            if idx < 2 or dist <= max_distance:
                retrieved_chunks.append({
                    "id": results['ids'][0][idx],
                    "content": results['documents'][0][idx],
                    "source": results['metadatas'][0][idx]['source'],
                    "distance": dist
                })

    context_text = "\n\n".join([f"[{c['source']} | {c['id']}]: {c['content']}" for
c in retrieved_chunks])

    prompt = f"""Використовуючи наведений нижче контекст, надай чітку та точну
відповідь на запитання користувача.
Якщо відповіді на запитання немає в наданому контексті, обов'язково повідом про це.

Контекст:
{context_text}

Запитання:
```

```
{query}
"""

    response = client.models.generate_content(
        model="gemini-3.5-flash-lite",
        contents=prompt,
    )
    return response.text.strip(), retrieved_chunks


if __name__ == "__main__":
    test_query = "Як працюють текстові данні в Python"


    print(f"ТЕСТОВЕ ЗАПИТАННЯ: \"{test_query}\"\n")

    print("[1] Відповідь LLM БЕЗ RAG (чиста модель):")
    ans_no_rag = answer_without_rag(test_query)
    print(ans_no_rag)


    print("[2] Відповідь LLM З ВИКОРИСТАННЯМ RAG (наша локальна база):")
    ans_rag, chunks = answer_with_rag(test_query)
    print(ans_rag)
```

[1] Відповідь LLM БЕЗ RAG (чиста модель):

Текстові дані в Python представлені за допомогою типу даних **`str`** (скорочення від *string* або «рядок»). Рядки в Python є надзвичайно потужними, зручними у використанні та мають велику кількість вбудованих інструментів для обробки.

Ось детальний розбір того, як працюють текстові дані в Python.

1. Створення рядків

Рядок створюється шляхом поміщення тексту всередину одинарних (`'`), подвійних (`"`) або потрійних (`"""` чи `'''`) лапок.

```python
name = 'Олег'  # одинарні лапки
city = "Київ"  # подвійні лапки
# Потрійні лапки використовуються для багаторядкового тексту
bio = """Мене звати Олег.
Я живу в Києві."""
```

2. Юнікод (Unicode) за замовчуванням

Це одна з найважливіших особливостей Python 3. Усі рядки за замовчуванням є **Unicode**-тестом. Це означає, що Python чудово розуміє кирилицю, ієрогліфи, емодзі та спеціальні символи без жодних додаткових налаштувань:

```python
greeting = "Привіт, світ! 🌍"
print(greeting)
```

3. Основні операції з рядками

* **Конкатенація (об'єднання):**

```python
first = "Hello"
second = "World"
result = first + " " + second  # "Hello World"
```

* **Множення (дублювання):**

```python
print("Ha" * 3)  # "HaHaHa"
```

* **Довжина рядка:**

За допомогою вбудованої функції `len()`.

```python
text = "Python"
print(len(text))  # 6
```

4. Індексація та зрізи (Slicing)

Текст у Python — це послідовність символів. Кожен символ має свій індекс, починаючи з **0**.

* **Індексація:**

```python
word = "Python"
print(word[0])  # 'P' (перший символ)
print(word[-1])  # 'n' (останній символ, з кінця)
```

* **Зрізи (отримання частини рядка):**

Снтаксис: `рядок[початок:кінець:крок]`

```python
text = "Програмно"
print(text[0:4])  # "Прог" (від 0 до 4 не включаючи 4)
print(text[:4])  # "Прог" (з початку до 4)
print(text[4:])  # "рамно" (від 4 до кінця)
print(text[::-1])  # "онмаргорП" (реверс рядка)
```

5. Незмінність (Immutability)

Це **найголовніше правило** щодо рядків у Python: **рядки незмінні**.

Це означає, що ви не можете змінити окремий символ у вже створеному рядку.

```python
text = "cat"
# text[0] = 'b'  # Викличе TypeError!
```

Якщо вам потрібно змінити текст, ви створюєте новий рядок на основі старого:

```python
text = "b" + text[1:]  # "bat"
```

```

### 6. Методи рядків

Python має десятки вбудованих методів для роботи з текстом. Оскільки рядки незмінні, ці методи не змінюють оригінальний рядок, а повертають новий.

* **Регістр:**

  ```python

  s = "привіт"

  print(s.upper())  # "ПРИВІТ"

  print("ОЛЕГ".lower())  # "олег"

  print("головна сторінка".title())  # "Головна Сторінка"

```

* **Пошук і заміна:**

  ```python

  text = "Я люблю Python"

  print(text.find("Python"))  # 8 (індекс початку слова)

  print(text.replace("люблю", "ненавижу"))  # "Я ненавиджу Python"

```

* **Очищення від зайвих пробілів:**

  ```python

  messy = "   текст з пробілами   "

  print(messy.strip())  # "текст з пробілами"

```

* **Розбиття та об'єднання (split / join):**

  ```python

  # Розбити рядок на список за розділювачем

  sentence = "яблуко,банан,пельмень"

  fruits = sentence.split(",")

  print(fruits)  # ['яблуко', 'банан', 'пельмень']

  # З'єднати список у рядок

  new_sentence = " - ".join(fruits)

  print(new_sentence)  # "яблуко - банан - пельмень"

```

7. Форматування тексту (Інтерполяція)

Щоб вставляти змінні всередину рядка, найсучаснішим і найзручнішим способом є **f-рядки (formatted string literals)**, які з'явилися в Python 3.6:

```python

name = "Марія"

age = 25

# Використання f-рядка

info = f"Мене звати {name}, і мені {age} років."

print(info)  # "Мене звати Марія, і мені 25 років."

# Всередині фігурних дужок можна навіть писати код:

print(f"Через 5 років мені буде {age + 5}.")

```

8. Екранування символів

Якщо вам потрібно використати спеціальні символи (наприклад, лапки всередині рядка або перенесення рядка), використовують зворотний слеш (`\`):

* `\n` — новий рядок

* `\t` — табуляція

* `\'` або `\"` — лапки всередину відповідних лапок

```python

text = "Він сказав: \"Привіт!\"\nІ пішов."

```

(Альтернатива для шляхів у файлах — "сирі" рядки, де слеші не екрануються: `r"C:\new_folder\text.txt"`)

Резюме

Працювати з текстом у Python дуже приємно завдяки інтуїтивному синтаксису, підтримці Unicode "з коробки" та багатій стандартній бібліотеці методів. Основне, що варто пам'ятати — рядки не можна змінити "на місці", але їх дуже легко трансформувати у нові рядки.

[2] Відповідь LLM З ВИКОРИСТАННЯМ RAG (наша локальна база):

На основі наданого контексту, текстові дані в Python працюють таким чином:

* Зберігаються у вигляді рядків (`str`), які за замовчуванням підтримують кодування Unicode.

* Рядки є **незмінними об'єктами**: після створення рядка неможливо модифікувати окремий символ.

* Для створення рядків можна використовувати одинарні, подвійні або потрійні лапки.

* Потрійні лапки зазвичай застосовуються для написання багаторядкових слів або документаційних рядків (`docstrings`).

* Порожні рядки в умовних інструкціях оцінюються як `False`, а непорожні — як `True`.

Критерій	LLM без використання RAG	LLM із використанням RAG
точність	Модель безпомилково описує синтаксис, Unicode, незмінність, зрізи та вбудовані методи Python.	Усі тези є правдивими та повністю збігаються з фактами із наданого чанку.
Повноту	Охоплює 8 підрозділів: індексація, зрізи, методи, f-рядки, екранування, приклади коду.	Висвітлює лише 5 конкретних тез, що були присутні у завантаженому `.txt` файлі.
відповідність наданим документам;	Модель не орієнтується на внутрішні файли та використовує загальні знання, отримані під час попереднього навчання.	Відповідь починається зі слів *«На основі наданого контексту...»* і містить тільки завантажені факти.
наявність вигаданої інформації.	існує теоретичний ризик їх виникнення при запитах щодо специфічних чи закритих бібліотек.	Модель не додала жодної деталі поза контекстом (наприклад, не описувала зрізи чи f-рядки, яких не було в чанку).
```